

DPP Assignment 2

Simon Vinding Brodersen(rzn875)

December 3, 2025

Benchmarking information

The benchmarks from this report are run on the "hendrixfut03ff" machine and on the GPU: "NVIDIA A100-PCIE-40GB". Further, the provided machine runs with a "AMD EPYC 7352 24-Core Processor".

1 Flattening with the New, More Efficient Rules

My code validates on default input (which is checking the implementation against the classic version) and the implementation is provided below:

```
def optimIi1Ker [m][n][q] (Sa: [m]u32, Da: [n]f32)
  (Sb: [m]u32, Db: [q]f32)
  (cs: [m]f32) (inds: [m]i64)
  : [m]f32 =
  let (Ba, flags'') = mkFlagArray Sa 0 (iota m)
  let flags' = map u32.i64 flags''
  let flags = map bool.u32 flags' :> [n]bool
  let flen_iot = reduce (+) 0 Sa |> i64.u32 |> iota :> [n]i64
  let iia1 = sgmScan (+) 0 flags (flags' :> [n]u32)
  let iia2 = map2(\i sgm -> (u32.i64 i) - Ba[i64.u32 sgm]) (flen_iot) iia1
  let (Bb, _) = mkFlagArray Sb false (replicate m true)
  let b_vals = map2 (\off ind -> Db[i64.u32 off + ind]) Bb inds
  let tmp1s = map2 (\a sgm -> (f32.sqrt a) * b_vals[i64.u32 sgm] + cs[i64.u32 sgm]) Da
    iia1
  let iot1s = iia2
  let iotfs = map f32.u32 iot1s
  let tmp2s = map2 (+) tmp1s iotfs
  let tmp_scan = sgmScan f32.max f32.lowest flags tmp2s
  let res = imap2 (iota m) Sa
    (\i s -> if s <= 0 then f32.lowest
      else if i == m-1
        then tmp_scan[n-1]
        else tmp_scan[i64.u32 Ba[i+1]-1]
    )
  in res
```

The runtime comparison between the standard classic implementation and the new optimized implementation can be seen below:

(m, q, p)	Classic (μs)	optimIII1 (μs)	Speedup
(10, 10 000 000, 100 000)	11212	7808	1.44
(100, 1 000 000, 10 000)	11192	7810	1.43
(1000, 100 000, 1000)	11250	7807	1.44
(10000, 10 000, 100)	11269	7866	1.45
(100000, 1000, 10)	11340	7908	1.43

Table 1: Performance comparison of classic and optimIII1 kernels with speedup.

This means that the new optimized implementation improves performance by around 1.44 across the board. This follows our general intuition about how the new flattening rules reduce the overall memory overhead needed and thus leads to performance improvements.

2 Flattening Rule for Scatter inside Map

Let Sx and Si denote the shape of the inner arrays of xss and iss same as S_{arr}^1 in the slides, let Dx denote the flat data of xss , let Di denote the flat data of iss and Dv be the flat data of vss . Then we would have the following flattening rules for the scatter inside a map:

```
let (Bx, _) = mkFlagArray Sx 0 (iota (length Sx))
let (Bi, flags') = mkFlagArray Si (iota (length Si))
let flags = map bool.u32 flags'
let iis = sgmScan (+) 0 flags flags'
let glob_idxxs = map2 (\sgm ind -> ind + Bx[sgm]) iis Di
in res = scatter Dx glob_idxxs Dv
```

$Bx[i]$ tells us the starting location of the inner list i in Dx . We then find the segment indices for the iss , such that we can add the offset, $Bx[sgm]$, to each of the inner lists of iss . This then gives us the final global index of each of the values. This then means we can directly scatter into Dx with the global indices and given each index corresponds to some value in vss , we don't have to change anything to Dv .

3 Flattening Rule for Histogram inside Map

Much the same as before, let Sx and Si the shape of the inner arrays of $histos$ and iss , same as S_{arr}^1 in the slides, let Dx denote the flat data of $histos$, let Di denote the flat data of iss and Dv be the flat data of vss .

Then we have the following flattening rules for the `reduce_by_index` inside a map:

```
let (Bx, _) = mkFlagArray Sx 0 (iota (length Sx))
let (Bi, flags') = mkFlagArray Si (iota (length Si))
let flags = map bool.u32 flags'
let iis = sgmScan (+) 0 flags flags'
let glob_idxxs = map2 (\sgm ind -> ind + Bx[sgm]) iis Di
in res = reduce_by_idx (⊙) 0⊙ Dx glob_idxxs Dv
```

The argument for correctness is analogous to that of the previous task.

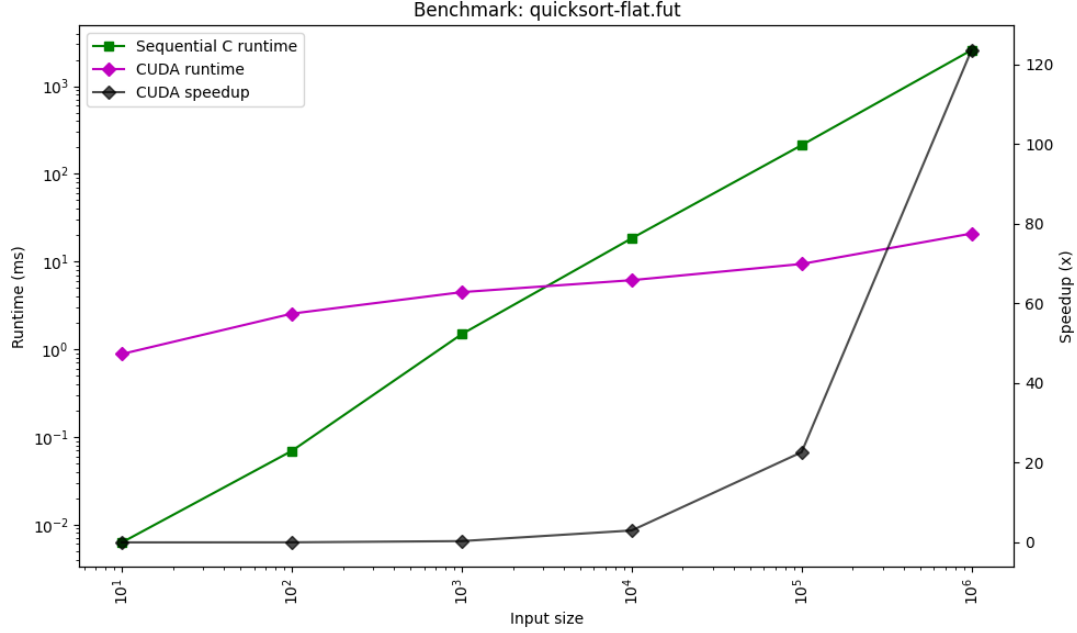


Figure 1: Benchmarking result of the baseline sequential C implementation of quicksort and with Cuda backend.

4 Lifted version of partition2

My implementation validates given the standard input. Further tests along with the implementation can be seen below.

```
-- ==
-- entry: main0
-- nobench input { [5, 5] [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] }
-- output { [3, 2] [5,5] [0, 2, 4, 1, 3, 6, 8, 5, 7, 9] }
-- nobench input { [10] [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] }
-- output { [5] [10] [0, 2, 4, 6, 8, 1, 3, 5, 7, 9] }
entry main0 [m][n] (shp: [m]i32) (arr: [n]i32) : ([m]i32, [m]i32, [n]i32) =
  let (ps, (shp', arr')) = partition2L (map (\x -> (x % 2) == 0i32) arr) 0i32 (shp, arr)
  in (ps, shp', arr')

-- futhark dataset -b --f32-bounds=-1000000.0:1000000.0 -g [10000000]f32 | ./quicksort-
-- flat -t /dev/stderr -r 2 > /dev/null
-- ==
-- entry: main
-- nobench compiled input { [29.0f32, 5.0f32, 7.0f32, 11.0f32, 2.0f32, 3.0f32, 13.0f32,
-- 23.0f32, 17.0f32, 19.0f32] }
-- output { [2.0f32, 3.0f32, 5.0f32, 7.0f32, 11.0f32, 13.0f32, 17.0f32, 19.0f32, 23.0f32,
-- 29.0f32] }
entry main [n] (arr: [n]f32) =
  let (_, res) = quicksortL ([i32.i64 n], arr)
  in res
```

```

def partition2L 't [n] [m]
  -- the shape of condsL is also shp
  (condsL: [n]bool)
  (dummy: t)
  (shp: [m]i32, arr: [n]t) : ([m]i32, ([m]i32, [n]t)) =
  -- These are just given, no rules are used
  let begs = scan (+) 0 shp
  let flags = mkFlagArray shp 0i32 (map (+ 1) (map i32.i64 (iota m))) :> [n]i32
  let outinds = sgmSumInt flags <| map (\f -> if f == 0 then 0 else f - 1) flags

  -- This uses the rule of map inside a map
  let tflgs = map (\c -> if c then 1i32 else 0i32) condsL
  let fflgs = map (\b -> 1 - b) tflgs

  -- This uses the rule for a scan inside a map
  let indsT = sgmSumInt flags tflgs
  let tmps = sgmSumInt flags fflgs

  -- This uses the map inside a map. However, as we depend on the (n>0) being
  -- the individual segments length, then we need need to take care of using
  -- the different n for index and the one for the > 0 check
  let tmp1s = map2 (\inc_n n -> if n > 0 then indsT[inc_n - 1] else -1) begs shp
  let indsF = map2 (\sgm item -> item + tmp1s[sgm]) outinds tmps

  -- map inside a map
  let inds = map3 (\c indT indF -> if c then indT - 1 else indF - 1) condsL indsT indsF

  -- My custom (and might I add beautiful) scatter rule from the previous section
  let glob_idx = map i64.i32 <| map2 (\sgm ind -> if sgm > 0 then ind + begs[sgm - 1]
    else ind) outinds inds
  let fltarrL = scatter (replicate n dummy) glob_idx arr
  in (tmp1s, (shp, fltarrL))

```

The comments correspond at an almost one to one the standard partition2 function and how I have managed to update the code. I have not added the partition2 code as well, as I hope the variable naming makes it easy to follow which parts have been translated from the original.

With this implementation the benchmarking results can be seen in Figure 1. Here we can see, how on small inputs the cost of loading the cuda kernels outweighs the benefits of using the GPU. As the input size grows, then we see that the CUDA backend outperforms the standard C implementation by a factor of about 120× at the largest part.

Further, it is interesting to note how the cuda does not have a completely flat curve initially. This supports our intuition, that the span of our implementation is $O(\lg n)$ as was found in the slides.